

Machine Learning-Based System

Cardiopulmonary Sound Separation

Team Members

Ashraf | Ahmad Akmal | Reshma | Sharwin

Focus

Working prototype
Architecture and components
Design patterns
Team reflection

Problem And Objectives

Mixed heart/lung sounds need a usable software workflow

Problem

Mixed recordings contain overlapping heart and lung sounds.

A standalone ML script lacks upload, storage, preview, and history.

Objective

Build a local FastAPI prototype using NeoSSNet.

Produce separated heart and lung WAV outputs.

Design objective

Use layered architecture, SQLite, SOLID principles, and design patterns.

System Scope And Features

Focused local prototype, not clinical diagnosis

Features

- Upload
- Validate
- Separate
- Preview
- Download
- History

Out of scope

No medical diagnosis, no clinical validation claim, no hospital deployment.

Software Architecture

Layered design

Layers

UI: HTML/CSS/JS

API: FastAPI routers

Business logic: services

ML: SeparationEngine/NeoSSNetStrategy

Data/storage: SQLite and local folders

Component-Level Design

Key components

Routers

UploadRouter, SeparationRouter

Services

SeparationService, ModelService,
StorageService, ResultService

ML and data

SeparationEngine, NeoSSNetStrategy,
SQLite models

Design Patterns Applied

Final clean mapping

Facade

SeparationRouter -> SeparationService ->
ModelService, StorageService,
ResultService

Strategy

SeparationEngine -> SeparationAlgorithm
-> NeoSSNetStrategy

Factory Method

UploadRouter -> AudioValidatorFactory ->
WavAudioValidatorFactory ->
WavAudioValidator

Prototype Workflow

Input to output

Flow

- Upload mixed WAV
- Validate file
- Select model
- Run separation
- Save heart/lung outputs
- Preview/download
- Record history

Solution Achieved And Future Work

Result and next steps

Achieved

- Working FastAPI prototype
- Real NeoSSNet separation
- SQLite traceability
- Preview/download/history

Future work

- Better UI progress
- More metrics
- Background processing
- More model strategies

Work Segregation

Team roles

Ashraf

Design coordination, backend/pattern alignment

Ahmad Akmal

UI and prototype flow

Reshma

Database, storage, requirements, SOLID

Sharwin

Diagrams, architecture, component design

Individual Self-Reflection

Contribution, challenge, improvement

Ashraf

Align code/UML/report; improve tests and deployment

Ahmad Akmal

UI simplicity; improve usability testing

Reshma

Database/history organization; improve metrics reporting

Sharwin

Clean diagrams; improve model/validator extension